

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
по курсу «Логика и основы алгоритмизации
в инженерных задачах»
на тему «Реализация алгоритма Форда-Беллмана»

Выполнил:
студент группы 24ВВВ3
Мингалёв М.А.

Принял:
Юрова О. В.

Содержание

Содержание.....	4
Реферат	5
Введение	6
1 Постановка задачи.....	9
2 Теоретическая часть задания	11
3.1 Основные функции и переменные программы.....	15
3.2 Функции	24
3.3 Главное меню программы.....	26
3.4 Операции с графом (пункты меню 1-6).....	26
3.5 Алгоритмические операции (пункты меню 7-9).....	27
3.6 Демонстрационные функции (пункты меню 10-11).....	27
3.7 Класс Graph.....	27
3.8 Алгоритмические методы класса Graph.....	28
3.9 Структура Edge.....	29
Заключение	35
Список используемых источников.....	36
ПриложениеА	37

Реферат

Отчет 46 страниц, 5 рисунков, 2 таблицы, 7 источников.

ГРАФ, ТЕОРИЯ ГРАФОВ, АЛГОРИТМ ФОРДА-БЕЛЛМАНА, КРАТЧАЙШИЙ ПУТЬ, ОТРИЦАТЕЛЬНЫЙ ЦИКЛ

Цель исследования – разработка программы, реализующей алгоритм Форда-Беллмана для нахождения кратчайших путей в графах различных типов, а также предоставляющей инструменты для создания, сохранения, загрузки и анализа графов.

В работе рассмотрен алгоритм Форда-Беллмана, который позволяет находить кратчайшие пути от одной вершины до всех остальных в графе с произвольными весами рёбер, включая отрицательные. Особое внимание уделено возможности обнаружения отрицательных циклов, что является ключевой особенностью данного алгоритма по сравнению с другими алгоритмами поиска кратчайших путей.

Введение

Алгоритм Форда-Беллмана — это алгоритм поиска кратчайших путей от одной вершины до всех остальных в графе, который позволяет работать с рёбрами отрицательного веса. Алгоритм используется для нахождения кратчайших расстояний во взвешенных графах и может обнаруживать отрицательные циклы. В отличие от алгоритма Дейкстры, который работает только с неотрицательными весами, алгоритм Форда-Беллмана применим к более широкому классу графов.

Основные характеристики алгоритма Форда-Беллмана:

- Работа с отрицательными весами: алгоритм корректно обрабатывает рёбра с отрицательным весом;
- Обнаружение отрицательных циклов: алгоритм выявляет наличие циклов с отрицательным суммарным весом, достижимых из начальной вершины;
- Временная сложность: $O(V \times E)$, где V — количество вершин, E — количество рёбер;
- Гибкость применения: может использоваться как для ориентированных, так и для неориентированных графов.

Структура реализации алгоритма включает следующие компоненты:

- Представление графа: в программе граф представлен в виде списка рёбер, где каждое ребро содержит начальную вершину, конечную вершину и вес;
- Инициализация расстояний: начальная вершина получает расстояние 0, все остальные — бесконечность;
- Релаксация рёбер: выполняется $(V-1)$ итерация, на каждой из которой происходит попытка улучшить расстояния до вершин;

- Проверка на отрицательные циклы: после (V-1) итераций выполняется дополнительная проверка — если расстояние можно улучшить, значит, в графе присутствует отрицательный цикл;
- Восстановление путей: для каждой вершины сохраняется предшественник, что позволяет восстановить сам путь, а не только его длину.

Реализованные функции в программе:

1. Класс Graph:

- Хранит информацию о графе: количество вершин, список рёбер, ориентированность и взвешенность;
- Метод addEdge() добавляет ребро в граф с учетом его типа (для неориентированного графа добавляется обратное ребро);
- Метод bellmanFord() является основной реализацией алгоритма Форда-Беллмана и возвращает два вектора: расстояния до вершин и предшественников;
- Метод getPath() восстанавливает путь от начальной до целевой вершины на основе вектора предшественников;
- Метод printInfo() выводит подробную информацию о структуре графа.

2. Функция demonstrateAllGraphTypes():

- Демонстрирует работу алгоритма на четырёх типах графов: ориентированный взвешенный, ориентированный невзвешенный, неориентированный взвешенный и неориентированный невзвешенный;
- Для каждого графа выводится его структура, результаты работы алгоритма (кратчайшие расстояния и пути) от заданной начальной вершины.

3. Функция testNegativeCycle():

- Специально тестирует случай графа с отрицательным циклом;
- Показывает, как алгоритм обнаруживает и сообщает о наличии отрицательного цикла.

В качестве среды разработки была выбрана Microsoft Visual Studio Community 2022, язык программирования – C++. Программа реализована как консольное приложение с использованием стандартной библиотеки C++.

Целью данной работы является разработка программы на языке C++, реализующей алгоритм Форда-Беллмана для поиска кратчайших путей в графах с возможностью обработки отрицательных весов и обнаружения отрицательных циклов. Программа демонстрирует работу алгоритма на различных типах графов и предоставляет наглядные результаты вычислений.

1 Постановка задачи

Алгоритм Форда-Беллмана должен работать с графом, представленным в виде множества вершин и направленных рёбер, каждое из которых имеет вес, определяющий стоимость перехода между вершинами. Состояние графа описывается матрицей смежности или списком рёбер, где каждое ребро характеризуется начальной вершиной, конечной вершиной и весом, который может быть положительным, отрицательным или нулевым.

Для поиска кратчайших путей используется алгоритм Форда-Беллмана. Его работа начинается с выбора стартовой вершины, которой присваивается расстояние 0, а всем остальным вершинам — бесконечность. На каждом шаге алгоритм выполняет релаксацию всех рёбер графа: если расстояние до конечной вершины можно уменьшить, пройдя через текущее ребро, то расстояние обновляется. Этот процесс повторяется $(V-1)$ раз, где V — количество вершин. Если после $(V-1)$ итераций возможно дальнейшее уменьшение расстояния, это свидетельствует о наличии в графе отрицательного цикла.

Алгоритм Форда-Беллмана может работать как с графами, сгенерированными программой, так и загруженными из файла. После получения графа необходимо вычислить кратчайшие пути, что осуществляется с помощью алгоритма Форда-Беллмана или может быть проверено вручную пользователем. Алгоритм Форда-Беллмана обрабатывает все рёбра графа, начиная со стартовой вершины, и последовательно обновляет расстояния до каждой достижимой вершины, пока не будут обработаны все вершины или не будет обнаружен отрицательный цикл. При ручной проверке пользователь может анализировать результаты работы алгоритма, представленные в виде таблицы расстояний и путей.

Алгоритм Форда-Беллмана строит маршруты, следуя которым можно определить кратчайшие расстояния от начальной вершины ко всем остальным вершинам графа. Он прекращает свою работу после выполнения $(V-1)$ итераций или при раннем обнаружении, что дальнейшая релаксация невозможна. Этот

метод эффективен для поиска кратчайших путей в графах с отрицательными весами и для обнаружения отрицательных циклов.

Программа должна предусматривать ввод параметров графа пользователем (выбор типа графа: ориентированный/неориентированный, взвешенный/невзвешенный, а также задание весов рёбер) и обеспечивать возможность сохранения и загрузки графа из файла. Управление программой осуществляется с помощью консольного интерфейса, что обеспечивает удобство взаимодействия пользователя с алгоритмом. Программа демонстрирует работу алгоритма на различных типах графов, включая графы с отрицательными циклами, и предоставляет подробную информацию о каждом шаге вычислений.

2 Теоретическая часть задания

Алгоритм Форда-Беллмана позволяет находить кратчайшие пути от одной вершины до всех остальных во взвешенном графе. Этот алгоритм основан на принципе релаксации рёбер и способен обрабатывать графы с отрицательными весами, а также обнаруживать отрицательные циклы.

```
1. ОРИЕНТИРОВАННЫЙ ВЗВЕШЕННЫЙ ГРАФ:
Тип графа: Ориентированный, Взвешенный
Количество вершин: 5
Количество ребер: 10
Ребра:
0 -> 1 (вес: 6)
0 -> 2 (вес: 7)
1 -> 2 (вес: 8)
1 -> 3 (вес: 5)
1 -> 4 (вес: -4)
2 -> 3 (вес: -3)
2 -> 4 (вес: 9)
3 -> 1 (вес: -2)
4 -> 0 (вес: 2)
4 -> 3 (вес: 7)

Кратчайшие расстояния от вершины 0:
До вершины 0: 0, путь: 0
До вершины 1: 2, путь: 0 -> 2 -> 3 -> 1
До вершины 2: 7, путь: 0 -> 2
До вершины 3: 4, путь: 0 -> 2 -> 3
До вершины 4: -2, путь: 0 -> 2 -> 3 -> 1 -> 4
```

Рисунок 1– Пример взвешенного графа

Алгоритм Форда-Беллмана позволяет находить кратчайшие пути в графах различной структуры, где веса рёбер могут быть как положительными, так и отрицательными, что делает его универсальным инструментом для решения задач маршрутизации.

Отличие алгоритма Форда-Беллмана от других алгоритмов поиска кратчайших путей (например, алгоритма Дейкстры) заключается в его способности работать с отрицательными весами. Алгоритм начинается с инициализации расстояний: стартовой вершине присваивается расстояние 0, всем остальным — бесконечность. Далее выполняется последовательная релаксация всех рёбер графа. Если расстояние до конечной вершины можно уменьшить через текущее ребро, оно обновляется.

Принцип работы алгоритма:

- Выбирается стартовая вершина, её расстояние устанавливается равным 0, расстояния до всех остальных вершин инициализируются как бесконечность (Рисунок 2). Создаётся массив предшественников для восстановления путей;

```
Кратчайшие расстояния от вершины 0:  
До вершины 0: 0, путь: 0  
До вершины 1: 1, путь: 0 -> 1  
До вершины 2: 1, путь: 0 -> 2  
До вершины 3: 2, путь: 0 -> 1 -> 3  
До вершины 4: 3, путь: 0 -> 1 -> 3 -> 4
```

Рисунок 2– Инициализация расстояний

- Выполняется (V-1) итераций, где V — количество вершин. На каждой итерации проверяются все рёбра графа. Если расстояние до конечной вершины можно улучшить через текущее ребро, происходит обновление расстояния и запись предшественника (Рисунок 3);

```
До вершины 1: 2, путь: 0 -> 2 -> 3 -> 1
```

Рисунок 3– Процесс релаксации рёбер

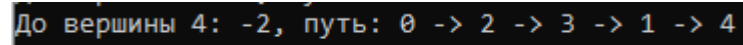
Этот процесс повторяется до тех пор, пока не будут выполнены все (V-1) итераций или пока на очередной итерации не останется изменений (оптимизация раннего выхода). После (V-1) итераций выполняется дополнительная проверка всех рёбер.

- Если расстояние можно ещё уменьшить, значит, в графе присутствует отрицательный цикл (Рисунок 4);

```
=== ТЕСТ С ОТРИЦАТЕЛЬНЫМ ЦИКЛОМ ===  
  
Тип графа: Ориентированный, Взвешенный  
Количество вершин: 4  
Количество ребер: 4  
Ребра:  
0 -> 1 (вес: 1)  
1 -> 2 (вес: -1)  
2 -> 3 (вес: -1)  
3 -> 1 (вес: -1)  
  
Граф содержит отрицательный цикл!  
Кратчайшие расстояния от вершины 0:  
До вершины 0: недостижима  
До вершины 1: недостижима  
До вершины 2: недостижима  
До вершины 3: недостижима
```

Рисунок 4– Отрицательный цикл

- Если отрицательных циклов нет, можно восстановить кратчайшие пути из стартовой вершины во все достижимые вершины с помощью массива предшественников (Рисунок 5);



```
До вершины 4: -2, путь: 0 -> 2 -> 3 -> 1 -> 4
```

Рисунок 5– Восстановление пути
по предшественникам

3 Описание программы

Программа была написана в среде разработки VisualStudioCommunity 2022 на языке программирования C++ с использованием стандартной библиотеки шаблонов (STL) и консольного интерфейса.

Далее будут приведены описания функций, структур и действия элементов управления, используемых в программе.

3.1 Основные функции и переменные программы

Для программной реализации алгоритма Форда-Беллмана для нахождения кратчайших путей в графе понадобятся четыре группы переменных и функций.

Первая группа отвечает за структуру данных графа и хранение информации о рёбрах. Для хранения информации о графе используется класс `Graph`, который содержит:

- `vertices` - количество вершин в графе
- `edges` - вектор рёбер типа `vector<Edge>`
- `isDirected` - флаг ориентированности графа
- `isWeighted` - флаг взвешенности графа

Каждое ребро представлено структурой `Edge`, содержащей информацию о начальной вершине (`from`), конечной вершине (`to`) и весе ребра (`weight`).

```
struct Edge {
    int from;
    int to;
    double weight;

    Edge(int f, int t, double w = 1.0) : from(f), to(t), weight(w) {}
};

class Graph {
private:
    int vertices;
    vector<Edge> edges;
    bool isDirected;
    bool isWeighted;

public:
    Graph(int v = 0, bool directed = true, bool weighted = true)
        : vertices(v), isDirected(directed), isWeighted(weighted) {}
}
```

`void addEdge(int from, int to, double weight = 1.0)` – Добавляет ребро в граф.

- Если граф невзвешенный, вес устанавливается в 1.0;
- Для неориентированного графа добавляется обратное ребро;
- Ребро сохраняется в векторе `edges`.

```

void addEdge(int from, int to, double weight = 1.0) {
    if (!isWeighted) weight = 1.0;

    edges.push_back(Edge(from, to, weight));

    if (!isDirected) {
        edges.push_back(Edge(to, from, weight));
    }
}

```

Вторая группа функций отвечает за реализацию алгоритма Форда-Беллмана. Функция `bellmanFord(int start)` находит кратчайшие пути от стартовой вершины до всех остальных.

- Инициализирует массивы расстояний (`dist`) и предшественников (`predecessor`);
- Выполняет $|V|-1$ итераций релаксации всех рёбер;
- Проверяет наличие отрицательных циклов;
- Возвращает пару: расстояния и предшественников.

```

pair<vector<double>, vector<int>> bellmanFord(int start) const {
    vector<double> dist(vertices, INF);
    vector<int> predecessor(vertices, -1);

    dist[start] = 0;

    for (int i = 0; i < vertices - 1; i++) {
        bool updated = false;

        for (const Edge& edge : edges) {
            if (dist[edge.from] < INF) {
                if (dist[edge.to] > dist[edge.from] + edge.weight) {
                    dist[edge.to] = dist[edge.from] + edge.weight;
                    predecessor[edge.to] = edge.from;
                    updated = true;
                }
            }
        }

        if (!updated) break;
    }

    bool hasNegativeCycle = false;
    for (const Edge& edge : edges) {
        if (dist[edge.from] < INF) {
            if (dist[edge.to] > dist[edge.from] + edge.weight) {
                hasNegativeCycle = true;
                break;
            }
        }
    }

    if (hasNegativeCycle) {
        cout << "Граф содержит отрицательный цикл!" << endl;
        fill(dist.begin(), dist.end(), INF);
        fill(predecessor.begin(), predecessor.end(), -1);
    }
}

```

```

    return { dist, predecessor };
}

```

Четвертая группа функций отвечает за вывод информации и демонстрацию работы алгоритма.

Функция `printInfo()` выводит полную информацию о графе.

- Тип графа (ориентированный/неориентированный, взвешенный/невзвешенный);
- Количество вершин и рёбер;
- Список всех рёбер с весами.

```

void printInfo() const {
    cout << "Тип графа: ";
    if (isDirected) cout << "Ориентированный";
    else cout << "Неориентированный";

    cout << ", ";

    if (isWeighted) cout << "Взвешенный";
    else cout << "Невзвешенный";

    cout << "\nКоличество вершин: " << vertices << endl;
    cout << "Количество ребер: " << edges.size() << endl;

    if (isDirected) {
        cout << "Ребра:\n";
        for (const Edge& edge : edges) {
            cout << edge.from << " -> " << edge.to;
            if (isWeighted) {
                cout << " (вес: " << edge.weight << ")";
            }
            cout << endl;
        }
    }
    else {
        cout << "Ребра (неориентированные):\n";
        vector<vector<bool>>> printed(vertices, vector<bool>(vertices, false));
        for (const Edge& edge : edges) {
            int u = min(edge.from, edge.to);
            int v = max(edge.from, edge.to);
            if (!printed[u][v]) {
                cout << u << " -- " << v;
                if (isWeighted) {
                    cout << " (вес: " << edge.weight << ")";
                }
                cout << endl;
                printed[u][v] = true;
            }
        }
    }
    cout << endl;
}

```

Функция `demonstrateAllGraphTypes()` демонстрирует работу алгоритма на четырёх типах графов:

1. Ориентированный взвешенный граф
2. Ориентированный невзвешенный граф
3. Неориентированный взвешенный граф
4. Неориентированный невзвешенный граф

Для каждого типа графа:

- Создаётся граф с тестовыми данными;
- Выводится информация о графе;
- Запускается алгоритм Форда-Беллмана;
- Выводятся кратчайшие расстояния и пути до всех вершин.

```
void demonstrateAllGraphTypes() {
    clearScreen();
    cout << "=== РАЗНЫЕ ТИПЫ ГРАФОВ И АЛГОРИТМ ФОРДА-БЕЛЛМАНА ===\n\n";

    vector<pair<string, Graph>> graphs = {
        {"ОРИЕНТИРОВАННЫЙ ВЗВЕШЕННЫЙ ГРАФ", Graph(5, true, true)},
        {"ОРИЕНТИРОВАННЫЙ НЕВЗВЕШЕННЫЙ ГРАФ", Graph(5, true, false)},
        {"НЕОРИЕНТИРОВАННЫЙ ВЗВЕШЕННЫЙ ГРАФ", Graph(5, false, true)},
        {"НЕОРИЕНТИРОВАННЫЙ НЕВЗВЕШЕННЫЙ ГРАФ", Graph(6, false, false)}
    };

    graphs[0].second.addEdge(0, 1, 6);
    graphs[0].second.addEdge(0, 2, 7);
    graphs[0].second.addEdge(1, 2, 8);
    graphs[0].second.addEdge(1, 3, 5);
    graphs[0].second.addEdge(1, 4, -4);
    graphs[0].second.addEdge(2, 3, -3);
    graphs[0].second.addEdge(2, 4, 9);
    graphs[0].second.addEdge(3, 1, -2);
    graphs[0].second.addEdge(4, 0, 2);
    graphs[0].second.addEdge(4, 3, 7);

    graphs[1].second.addEdge(0, 1);
    graphs[1].second.addEdge(0, 2);
    graphs[1].second.addEdge(1, 2);
    graphs[1].second.addEdge(1, 3);
    graphs[1].second.addEdge(2, 3);
    graphs[1].second.addEdge(3, 4);

    graphs[2].second.addEdge(0, 1, 2);
    graphs[2].second.addEdge(0, 2, 3);
    graphs[2].second.addEdge(1, 2, 1);
    graphs[2].second.addEdge(1, 3, 4);
    graphs[2].second.addEdge(2, 4, 5);
    graphs[2].second.addEdge(3, 4, 1);
}
```



```

graphs[3].second.addEdge(0, 1);
graphs[3].second.addEdge(0, 2);
graphs[3].second.addEdge(1, 3);
graphs[3].second.addEdge(2, 3);
graphs[3].second.addEdge(3, 4);
graphs[3].second.addEdge(4, 5);

for (int i = 0; i < graphs.size(); i++) {
    cout << i + 1 << ". " << graphs[i].first << ":\n";
    graphs[i].second.printInfo();

    auto result = graphs[i].second.bellmanFord(0);
    vector<double> distances = result.first;
    vector<int> predecessor = result.second;

    graphs[i].second.printDetailedPaths(0, distances, predecessor);
    cout << "\n" << string(60, '-') << "\n\n";
}

cout << "Нажмите Enter для продолжения...";
cin.ignore();
cin.get();
}

```

Функция testNegativeCycle() демонстрирует работу алгоритма на графе с отрицательным циклом.

- Создаётся граф с отрицательным циклом;
- Запускается алгоритм Форда-Беллмана;
- Выводится сообщение об обнаружении отрицательного цикла;
- Все расстояния сбрасываются в INF.

```

void testNegativeCycle() {
    clearScreen();
    cout << "=== ТЕСТ С ОТРИЦАТЕЛЬНЫМ ЦИКЛОМ ===\n\n";

    Graph g(4, true, true);
    g.addEdge(0, 1, 1);
    g.addEdge(1, 2, -1);
    g.addEdge(2, 3, -1);
    g.addEdge(3, 1, -1);

    g.printInfo();

    auto result = g.bellmanFord(0);
    vector<double> distances = result.first;
    vector<int> predecessor = result.second;

    g.printDetailedPaths(0, distances, predecessor);

    cout << "\nНажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
}

```

Основная функция main() организует работу всей программы:

- Устанавливает локаль для поддержки русского языка;

- Запускает демонстрацию работы на всех типах графов;
- Тестирует работу алгоритма на графе с отрицательным циклом.

```
int main() {
    setlocale(LC_ALL, "rus");
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    srand(time(NULL));

    int choice;
    do {
        clearScreen();
        printMenu();
        cin >> choice;
        clearScreen();

        switch (choice) {
            case 1: {
                int vertices;
                bool directed, weighted;
                char dirChoice, weightChoice;

                cout << "" << endl;
                cout << "        ЗАДАНИЕ ПАРАМЕТРОВ ГРАФА        " << endl;
                cout << "" << endl;

                cout << "Введите количество вершин: ";
                cin >> vertices;

                cout << "Ориентированный граф? (y/n): ";
                cin >> dirChoice;
                directed = (dirChoice == 'y' || dirChoice == 'Y');

                cout << "Взвешенный граф? (y/n): ";
                cin >> weightChoice;
                weighted = (weightChoice == 'y' || weightChoice == 'Y');

                currentGraph.setVertices(vertices);
                currentGraph.setDirected(directed);
                currentGraph.setWeighted(weighted);
                currentGraph.clearEdges();

                graphCreated = true;
                cout << "\nПараметры графа установлены!" << endl;
                cout << "Нажмите Enter для продолжения...";
                cin.ignore();
                cin.get();
                break;
            }

            case 2: {
                if (!graphCreated) {
                    cout << "Сначала задайте параметры графа (пункт 1)!" << endl;
                }
                else {
                    currentGraph.clearEdges();
                    currentGraph.generateRandomGraph();
                    cout << "\nГраф сгенерирован успешно!" << endl;
                }
                cout << "Нажмите Enter для продолжения...";
                cin.ignore();
                cin.get();
            }
        }
    }
}
```

```

        break;
    }

    case 3: {
        currentGraph.generateCompleteRandomGraph();
        graphCreated = true;
        cout << "\nНажмите Enter для продолжения...";
        cin.ignore();
        cin.get();
        break;
    }

    case 4: {
        if (!graphCreated) {
            cout << "Сначала задайте параметры графа (пункт 1)!" << endl;
        }
        else {
            currentGraph.clearEdges();
            currentGraph.inputFromKeyboard();
            cout << "\nГраф введен успешно!" << endl;
        }
        cout << "Нажмите Enter для продолжения...";
        cin.ignore();
        cin.get();
        break;
    }

    case 5: {
        string filename;
        cout << "Введите имя файла для загрузки: ";
        cin >> filename;
        currentGraph.loadFromFile(filename);
        graphCreated = true;
        cout << "Нажмите Enter для продолжения...";
        cin.ignore();
        cin.get();
        break;
    }

    case 6: {
        if (!graphCreated) {
            cout << "Граф еще не создан!" << endl;
        }
        else {
            currentGraph.printInfo();
        }
        cout << "Нажмите Enter для продолжения...";
        cin.ignore();
        cin.get();
        break;
    }

    case 7: {
        if (!graphCreated) {
            cout << "Граф еще не создан!" << endl;
        }
        else {
            int startVertex;
            cout << "Введите начальную вершину (0-" << currentGraph.getVerticesCount() - 1 << "): ";
            cin >> startVertex;

            if (startVertex < 0 || startVertex >= currentGraph.getVerticesCount()) {
                cout << "Неверная вершина!" << endl;
            }
            else {
                auto result = currentGraph.bellmanFord(startVertex);
                lastDistances = result.first;
            }
        }
    }
}

```

```

        lastPredecessor = result.second;
        lastStartVertex = startVertex;

        currentGraph.printDetailedPaths(startVertex, lastDistances, lastPredecessor);
    }
}
cout << "\nНажмите Enter для продолжения...";
cin.ignore();
cin.get();
break;
}

case 8: {
    if (!graphCreated) {
        cout << "Граф еще не создан!" << endl;
    }
    else {
        string filename;
        cout << "Введите имя файла для сохранения: ";
        cin >> filename;
        currentGraph.saveToFile(filename);
    }
    cout << "Нажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}

case 9: {
    if (!graphCreated || lastStartVertex == -1) {
        cout << "Сначала выполните алгоритм (пункт 7)!" << endl;
    }
    else {
        string filename;
        cout << "Введите имя файла для сохранения результатов: ";
        cin >> filename;
        saveResultsToFile(currentGraph, lastDistances, lastPredecessor, lastStartVertex, filename);
    }
    cout << "Нажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}

case 10: {
    demonstrateAllGraphTypes();
    break;
}

case 11: {
    testNegativeCycle();
    break;
}

case 0: {
    cout << "Выход из программы..." << endl;
    break;
}

default: {
    cout << "Неверный выбор! Попробуйте снова." << endl;
    cout << "Нажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}
}

```

```
    } while (choice != 0);  
    return 0;  
}
```

3.2 Функции

`int main()` – Главная функция программы, которая устанавливает локаль для поддержки русского языка, вызывает функцию `demonstrateAllGraphTypes()` для демонстрации работы алгоритма на всех типах графов и функцию `testNegativeCycle()` для тестирования работы алгоритма на графе с отрицательным циклом.

`Graph::Graph(int v, bool directed = true, bool weighted = true)` – Конструктор класса `Graph`, инициализирующий граф с указанным количеством вершин, направленностью и взвешенностью.

`void Graph::addEdge(int from, int to, double weight = 1.0)` – Метод для добавления ребра в граф, который принимает начальную и конечную вершины, а также вес ребра. Для невзвешенных графов вес автоматически устанавливается в 1.0, для неориентированных графов добавляется обратное ребро.

`pair<vector<double>, vector<int>> Graph::bellmanFord(int start)` – Основной метод, реализующий алгоритм Форда-Беллмана для нахождения кратчайших путей от указанной стартовой вершины до всех остальных. Возвращает пару, содержащую расстояния до всех вершин и предшественников для восстановления путей.

`vector<int> Graph::getPath(const vector<int>& predecessor, int start, int target)` – Вспомогательный метод для восстановления кратчайшего пути от стартовой до целевой вершины на основе массива предшественников, полученного из алгоритма Форда-Беллмана.

`void Graph::printInfo()` – Метод для вывода полной информации о графе, включая его тип (ориентированный/неориентированный, взвешенный/невзвешенный), количество вершин и рёбер, а также список всех рёбер с их весами.

`void demonstrateAllGraphTypes()` – Демонстрационная функция, которая тестирует алгоритм Форда-Беллмана на четырёх различных типах графов:

ориентированном взвешенном, ориентированном невзвешенном, неориентированном взвешенном и неориентированном невзвешенном.

`void testNegativeCycle()` – Тестовая функция, которая создаёт граф с отрицательным циклом и демонстрирует работу алгоритма Форда-Беллмана при обнаружении такого цикла, включая вывод соответствующего предупреждения и сброс результатов.

`Edge::Edge(int f, int t, double w = 1.0)` – Конструктор структуры `Edge`, инициализирующий ребро с указанными начальной вершиной, конечной вершиной и весом (по умолчанию 1.0).

3.3 Главное меню программы

`printMenu()` – Выводит на экран главное меню программы, которое содержит 12 пунктов для взаимодействия с пользователем. Отображает название программы "АЛГОРИТМ ФОРДА-БЕЛЛМАНА" и список доступных опций с номерами от 0 до 11.

`main()` - основной цикл программы – Обрабатывает выбор пользователя из главного меню и вызывает соответствующие функции. Цикл продолжается до тех пор, пока пользователь не выберет пункт 0 (Выход). Каждый выбор очищает экран и выполняет соответствующую операцию.

3.4 Операции с графом (пункты меню 1-6)

Пункт 1: Задать параметры графа – Позволяет пользователю задать основные параметры графа: количество вершин, тип графа (ориентированный/неориентированный) и взвешенность. После установки параметров флаг `graphCreated` устанавливается в `true`.

Пункт 2: Автоматическая генерация графа – Генерирует случайный граф на основе ранее заданных параметров. Использует метод `generateRandomGraph()` для создания рёбер со случайными весами в диапазоне от -10.0 до 10.0.

Пункт 3: Случайная генерация графа – Предоставляет расширенный интерфейс для создания полностью случайного графа. Пользователь может задать количество вершин, тип графа, взвешенность, диапазон весов и плотность графа (разреженный, средней плотности, плотный).

Пункт 4: Ввод графа с клавиатуры – Позволяет пользователю вручную ввести рёбра графа. Для каждого ребра запрашиваются начальная и конечная вершины, а также вес (если граф взвешенный). Проверяет корректность введённых данных.

Пункт 5: Загрузить граф из файла – Загружает ранее сохранённый граф из текстового файла. Файл должен содержать количество вершин, тип графа, взвешенность и список рёбер с весами.

Пункт 6: Показать информацию о графе – Отображает подробную информацию о текущем графе, включая тип, количество вершин и рёбер, а также полный список всех рёбер с их весами.

3.5 Алгоритмические операции (пункты меню 7-9)

Пункт 7: Выполнить алгоритм Форда-Беллмана – Запускает алгоритм Форда-Беллмана для нахождения кратчайших путей от указанной начальной вершины. Запрашивает у пользователя начальную вершину, выполняет алгоритм и выводит подробные результаты, включая расстояния и полные пути до всех вершин.

Пункт 8: Сохранить граф в файл – Сохраняет текущий граф в текстовый файл. Сохраняются все параметры графа и список рёбер, что позволяет позже загрузить граф для дальнейшей работы.

Пункт 9: Сохранить результаты в файл – Сохраняет результаты последнего выполнения алгоритма Форда-Беллмана в файл. Включает параметры графа, начальную вершину и подробные пути до всех достижимых вершин.

3.6 Демонстрационные функции (пункты меню 10-11)

Пункт 10: Демонстрация всех типов графов – Запускает демонстрационный режим, который последовательно тестирует алгоритм на четырёх различных типах графов. Каждый тип графа демонстрируется с полной информацией и результатами работы алгоритма.

Пункт 11: Тест с отрицательным циклом – Демонстрирует работу алгоритма на графе, содержащем отрицательный цикл. Показывает, как алгоритм обнаруживает отрицательный цикл и обрабатывает эту ситуацию.

3.7 Класс Graph

```
class Graph {  
private:  
    int vertices;
```

```
vector<Edge> edges;  
bool isDirected;  
bool isWeighted;
```

Класс `Graph` – Основной класс программы, представляющий граф и содержащий все методы для работы с ним.

- `Graph(int v, bool directed, bool weighted)` – Конструктор класса, инициализирующий граф с заданным количеством вершин, направленностью и взвешенностью;
- `setVertices(int v)` – Устанавливает количество вершин в графе;
- `setDirected(bool directed)` – Устанавливает тип графа (ориентированный/неориентированный);
- `setWeighted(bool weighted)` – Устанавливает взвешенность графа;
- `addEdge(int from, int to, double weight)` – Добавляет ребро в граф с указанными параметрами;
- `clearEdges()` – Очищает список всех рёбер графа;
- `getVerticesCount()` – Возвращает количество вершин в графе;
- `getIsDirected()` – Возвращает тип графа;
- `getIsWeighted()` – Возвращает взвешенность графа;
- `getEdges()` – Возвращает список всех рёбер графа.

3.8 Алгоритмические методы класса `Graph`

`pair<vector<double>, vector<int>> bellmanFord(int start)` – Основной метод, реализующий алгоритм Форда-Беллмана. Находит кратчайшие пути от указанной вершины до всех остальных, проверяет наличие отрицательных циклов и возвращает пару массивов: расстояний и предшественников.

`vector<int> getPath(const vector<int>& predecessor, int start, int target)` – Восстанавливает кратчайший путь от начальной до целевой вершины на основе массива предшественников, полученного из алгоритма Форда-Беллмана.

`void printDetailedPaths(int start, const vector<double>& distances, const vector<int>& predecessor)` – Выводит подробную информацию о кратчайших

путях от начальной вершины до всех остальных, включая расстояния и полные пути.

`void saveToFile(const string& filename)` – Сохраняет граф в текстовый файл в формате, который может быть позже загружен.

`void loadFromFile(const string& filename)` – Загружает граф из текстового файла, восстанавливая все параметры и рёбра.

`void generateRandomGraph(int maxEdges, double minWeight, double maxWeight)` – Генерирует случайный граф с заданными параметрами. Создаёт случайные рёбра между вершинами с весами в указанном диапазоне.

`void generateCompleteRandomGraph()` – Предоставляет интерактивный интерфейс для создания полностью случайного графа с настраиваемыми параметрами.

`void inputFromKeyboard()` – Позволяет пользователю ввести граф вручную через консольный интерфейс.

3.9 Структура Edge

```
struct Edge {
```

```
    int from;
```

```
    int to;
```

```
    double weight;
```

```
    Edge(int f, int t, double w = 1.0) : from(f), to(t), weight(w) {}
```

```
};
```

Структура Edge – Представляет одно ребро графа с его основными характеристиками.

- from – Начальная вершина ребра;
- to – Конечная вершина ребра;
- weight – Вес ребра (по умолчанию 1.0 для невзвешенных графов);
- Edge(int f, int t, double w) – Конструктор структуры, инициализирующий все поля.

Основные компоненты программы

`main()` – Главная функция программы, которая инициализирует окружение, запускает демонстрационные тесты и управляет выполнением всей программы.

`demonstrateAllGraphTypes()` – Демонстрационный модуль, который запускает последовательное тестирование алгоритма на четырёх различных типах графов, обеспечивая наглядное представление его работы.

`testNegativeCycle()` – Специальный тестовый модуль для проверки обработки графов с отрицательными циклами, демонстрирующий корректность работы алгоритма в исключительных ситуациях.

Класс `Graph` и его методы

`Graph::addEdge()` – Метод для добавления рёбер в граф. Обрабатывает особенности неориентированных графов (автоматическое добавление обратного ребра) и невзвешенных графов (установка веса по умолчанию).

`Graph::bellmanFord()` – Основной алгоритмический метод, реализующий алгоритм Форда-Беллмана для нахождения кратчайших путей от заданной вершины до всех остальных.

`Graph::getPath()` – Вспомогательный метод для восстановления конкретного пути между двумя вершинами на основе результатов алгоритма.

`Graph::printInfo()` – Метод диагностики, выводящий подробную информацию о структуре графа для отладки и понимания текущего состояния.

Вспомогательные структуры данных

`Edge` структура – Базовый элемент графа, содержащий информацию о соединении между вершинами и весе этого соединения.

`Graph` конструктор – Инициализирует объект графа с заданными параметрами: количеством вершин, направленностью и взвешенностью.

Демонстрационные сценарии

Ориентированный взвешенный граф сценарий – Тестирование алгоритма на сложном ориентированном графе с различными весами рёбер, включая отрицательные веса.

Ориентированный невзвешенный граф сценарий** – Демонстрация работы алгоритма на ориентированном графе, где все рёбра имеют одинаковый вес.

Неориентированный взвешенный граф сценарий – Тестирование на неориентированном графе с разными весами рёбер.

Неориентированный невзвешенный граф сценарий – Демонстрация работы алгоритма на простейшем типе графа.

Тест с отрицательным циклом – Специальный сценарий для проверки обнаружения и обработки отрицательных циклов.

Структура Edge

```
struct Edge {  
    int from;  
    int to;  
    double weight;  
  
    Edge(int f, int t, double w = 1.0) : from(f), to(t), weight(w) {}  
};
```

struct Edge – Структура, представляющая ребро графа. Каждое ребро содержит информацию о начальной и конечной вершинах, а также весе соединения.

- from – Начальная вершина ребра, откуда исходит соединение;
- to – Конечная вершина ребра, куда ведёт соединение;
- weight – Вес ребра, используемый при расчёте кратчайших путей;
- Edge(int f, int t, double w = 1.0) – Конструктор структуры Edge, который инициализирует ребро с заданными параметрами соединения и весом.

Класс Graph

Класс Graph – Основной класс программы, представляющий граф и содержащий все методы для работы с ним. Класс включает в себя:

- vertices – Количество вершин в графе;
- edges – Коллекция всех рёбер графа;
- isDirected – Флаг, указывающий, является ли граф ориентированным;
- isWeighted – Флаг, указывающий, является ли граф взвешенным.

При создании объекта Graph можно задать:

- Количество вершин для работы алгоритма;
- Направленность графа (ориентированный или неориентированный);
- Взвешенность графа (наличие весов у рёбер).

Таблица 1. – Описание поведения программы при тестировании

№	Описание теста	Предусловие	Тестирование	Ожидаемый результат
1	Запуск программы	нет	Запуск программы с помощью Visual Studio или исполняемого файла	Корректный запуск программы, вывод заголовка и начало демонстрации работы алгоритма
2	Тестирование ориентированного взвешенного графа	Программа запущена	Выполнение тестового сценария с ориентированным взвешенным графом	Корректное создание объекта класса Graph с заданными параметрами, правильное выполнение алгоритма Форда-Беллмана, вывод кратчайших расстояний и путей от стартовой вершины
3	Тестирование ориентированного невзвешенного графа	Программа запущена	Выполнение тестового сценария с ориентированным невзвешенным графом	Успешное создание графа без весов рёбер, корректная работа алгоритма с единичными весами, правильный вывод минимального количества шагов до каждой вершины
4	Тестирование неориентированного взвешенного графа	Программа запущена	Выполнение тестового сценария с неориентированным взвешенным графом	Успешное создание неориентированного графа, корректная обработка рёбер в обоих направлениях, правильный расчёт кратчайших путей в графе без направления
5	Тестирование неориентированного невзвешенного графа	Программа запущена	Выполнение тестового сценария с неориентированным невзвешенным графом	Успешное создание простейшего типа графа, корректный поиск кратчайших путей как минимального количества рёбер, правильный вывод результатов
6	Тестирование графа с отрицательным циклом	Программа запущена	Выполнение теста с графом, содержащим отрицательный цикл	Корректное обнаружение отрицательного цикла, вывод соответствующего предупреждения, сброс результатов расстояний в состояние INF, предотвращение некорректных вычислений

Таблица 2. – Дополнительные тесты алгоритмической корректности

№	Описание теста	Предусловие	Тестирование	Ожидаемый результат
1	Тестирование недостижимых вершин	Программа запущена	Создание графа с изолированными вершинами	Корректное определение недостижимых вершин, вывод расстояния INF, отсутствие путей к этим вершинам
2	Тестирование восстановления	Программа запущена	Запуск алгоритма на различных графах с	Правильное восстановление путей от стартовой до целевых вершин, корректная

	путей		последующим восстановлением конкретных путей	обработка случаев отсутствия пути
3	Тестирование информации о графе	Программа запущена	Вызов метода printInfo() на различных типах графов	Полный и корректный вывод информации о структуре графа, типах рёбер, весах и количестве элементов
4	Тестирование обработки больших графов	Программа запущена	Создание графов с большим количеством вершин и рёбер	Стабильная работа программы без утечек памяти, корректные результаты вычислений в разумное время

Заключение

При выполнении данной курсовой работы были получены навыки разработки алгоритмических программ на языке C++. Была освоена работа с различными структурами данных, изучены методы реализации алгоритмов поиска кратчайших путей в графах. Также были получены основные навыки отладки и тестирования алгоритмических программ и программирования в среде Microsoft Visual Studio 2022 на языке C++.

В рамках курсовой работы была разработана программа для нахождения кратчайших путей в графах с использованием алгоритма Форда-Беллмана. Программа имеет все необходимые функции для работы с различными типами графов: ориентированными и неориентированными, взвешенными и невзвешенными. Реализована корректная обработка граничных случаев, включая обнаружение отрицательных циклов и работу с недостижимыми вершинами.

В дальнейшем программу можно улучшить, путём добавления графического интерфейса с визуализацией графов и найденных путей, что позволит наглядно демонстрировать работу алгоритма. Также можно расширить функциональность программы добавлением других алгоритмов поиска кратчайших путей (Дейкстры, Флойда-Уоршелла), поддержкой работы с большими графами из файлов и реализацией интерактивного построения графов пользователем.

Список используемых источников

1. Кормен, Т. Алгоритмы: построение и анализ / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. – 3-е изд. – М. : Вильямс, 2013. – 1328 с.
2. Седжвик, Р. Алгоритмы на C++. Части 1–4: Фундаментальные алгоритмы, структуры данных, сортировка, поиск / Р. Седжвик. – 3-е изд. – К. : ДияСофт, 2002. – 688 с.
3. Страуструп, Б. Язык программирования C++ / Б. Страуструп. – 4-е изд. – М. : Вильямс, 2015. – 1136 с.
4. Прата, С. Язык программирования C++ (6-е издание) : лекции и упражнения / С. Прата. – М. : Вильямс, 2012. – 1184 с.
5. Ахо, А. Структуры данных и алгоритмы / А. Ахо, Дж. Хопкрофт, Дж. Ульман. – М. : Вильямс, 2000. – 384 с.
6. Официальная документация по стандарту C++ [Электронный ресурс]. – Режим доступа: <https://isocpp.org> (дата обращения: 25.12.2025).
7. Документация Microsoft по языку C++ в Visual Studio [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/cpp/> (дата обращения: 25.12.2025).

ПриложениеА

Листинги программы

Файлы main.cpp

```
#define _CRT_SECURE_NO_WARNINGS

#include <iostream>
#include <vector>
#include <limits>
#include <algorithm>
#include <clocale>
#include <cstdlib>
#include <ctime>
#include <fstream>
#include <string>
#include <windows.h>
#include <cmath>

using namespace std;

const double INF = numeric_limits<double>::infinity();

class Graph;

extern Graph currentGraph;
extern vector<double> lastDistances;
extern vector<int> lastPredecessor;
extern int lastStartVertex;
extern bool graphCreated;

int main() {
    setlocale(LC_ALL, "rus");
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    srand(time(NULL));

    int choice;
    do {
        clearScreen();
        printMenu();
        cin >> choice;
        clearScreen();

        switch (choice) {
            case 1: {
                int vertices;
                bool directed, weighted;
                char dirChoice, weightChoice;

                cout << "" << endl;
                cout << "        ЗАДАНИЕ ПАРАМЕТРОВ ГРАФА        " << endl;
                cout << "" << endl;

                cout << "Введите количество вершин: ";
                cin >> vertices;

                cout << "Ориентированный граф? (y/n): ";
                cin >> dirChoice;
                directed = (dirChoice == 'y' || dirChoice == 'Y');

                cout << "Взвешенный граф? (y/n): ";
                cin >> weightChoice;
                weighted = (weightChoice == 'y' || weightChoice == 'Y');
                currentGraph.setVertices(vertices);
```

```

currentGraph.setDirected(directed);
currentGraph.setWeighted(weighted);
currentGraph.clearEdges();

graphCreated = true;
cout << "\nПараметры графа установлены!" << endl;
cout << "Нажмите Enter для продолжения...";
cin.ignore();
cin.get();
break;
}

case 2: {
    if (!graphCreated) {
        cout << "Сначала задайте параметры графа (пункт 1)!" << endl;
    }
    else {
        currentGraph.clearEdges();
        currentGraph.generateRandomGraph();
        cout << "\nГраф сгенерирован успешно!" << endl;
    }
    cout << "Нажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}

case 3: {
    currentGraph.generateCompleteRandomGraph();
    graphCreated = true;
    cout << "\nНажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}

case 4: {
    if (!graphCreated) {
        cout << "Сначала задайте параметры графа (пункт 1)!" << endl;
    }
    else {
        currentGraph.clearEdges();
        currentGraph.inputFromKeyboard();
        cout << "\nГраф введен успешно!" << endl;
    }
    cout << "Нажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}

case 5: {
    string filename;
    cout << "Введите имя файла для загрузки: ";
    cin >> filename;
    currentGraph.loadFromFile(filename);
    graphCreated = true;
    cout << "Нажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}

case 6: {
    if (!graphCreated) {
        cout << "Граф еще не создан!" << endl;
    }
}

```

```

else {
    currentGraph.printInfo();
}
cout << "Нажмите Enter для продолжения...";
cin.ignore();
cin.get();
break;
}

case 7: {
    if (!graphCreated) {
        cout << "Граф еще не создан!" << endl;
    }
    else {
        int startVertex;
        cout << "Введите начальную вершину (0-" << currentGraph.getVerticesCount() - 1 << "): ";
        cin >> startVertex;

        if (startVertex < 0 || startVertex >= currentGraph.getVerticesCount()) {
            cout << "Неверная вершина!" << endl;
        }
        else {
            auto result = currentGraph.bellmanFord(startVertex);
            lastDistances = result.first;
            lastPredecessor = result.second;
            lastStartVertex = startVertex;

            currentGraph.printDetailedPaths(startVertex, lastDistances, lastPredecessor);
        }
    }
    cout << "\nНажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}

case 8: {
    if (!graphCreated) {
        cout << "Граф еще не создан!" << endl;
    }
    else {
        string filename;
        cout << "Введите имя файла для сохранения: ";
        cin >> filename;
        currentGraph.saveToFile(filename);
    }
    cout << "Нажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}

case 9: {
    if (!graphCreated || lastStartVertex == -1) {
        cout << "Сначала выполните алгоритм (пункт 7)!" << endl;
    }
    else {
        string filename;
        cout << "Введите имя файла для сохранения результатов: ";
        cin >> filename;
        saveResultsToFile(currentGraph, lastDistances, lastPredecessor, lastStartVertex, filename);
    }
    cout << "Нажмите Enter для продолжения...";
    cin.ignore();
    cin.get();
    break;
}

```

```

    case 10: {
        demonstrateAllGraphTypes();
        break;
    }

    case 11: {
        testNegativeCycle();
        break;
    }

    case 0: {
        cout << "Выход из программы..." << endl;
        break;
    }

    default: {
        cout << "Неверный выбор! Попробуйте снова." << endl;
        cout << "Нажмите Enter для продолжения...";
        cin.ignore();
        cin.get();
        break;
    }
} while (choice != 0);

return 0;
}

```

Файл Graph.cpp

```
#define _CRT_SECURE_NO_WARNINGS
```

```

#include <iostream>
#include <vector>
#include <limits>
#include <algorithm>
#include <locale>
#include <cstdlib>
#include <ctime>
#include <fstream>
#include <string>
#include <windows.h>
#include <cmath>

using namespace std;

class Graph {
private:
    int vertices;
    vector<Edge> edges;
    bool isDirected;
    bool isWeighted;

public:
    Graph(int v = 0, bool directed = true, bool weighted = true)
        : vertices(v), isDirected(directed), isWeighted(weighted) {
    }
    void setVertices(int v) {
        vertices = v;
        edges.clear();
    }

    void setDirected(bool directed) {
        isDirected = directed;
    }
}

```

```

void setWeighted(bool weighted) {
    isWeighted = weighted;
}

void addEdge(int from, int to, double weight = 1.0) {
    if (!isWeighted) weight = 1.0;

    edges.push_back(Edge(from, to, weight));

    if (!isDirected) {
        edges.push_back(Edge(to, from, weight));
    }
}

void clearEdges() {
    edges.clear();
}

int getVerticesCount() const {
    return vertices;
}

bool getIsDirected() const {
    return isDirected;
}

bool getIsWeighted() const {
    return isWeighted;
}

const vector<Edge>& getEdges() const {
    return edges;
}

pair<vector<double>, vector<int>> bellmanFord(int start) const {
    vector<double> dist(vertices, INF);
    vector<int> predecessor(vertices, -1);

    dist[start] = 0;

    for (int i = 0; i < vertices - 1; i++) {
        bool updated = false;

        for (const Edge& edge : edges) {
            if (dist[edge.from] < INF) {
                if (dist[edge.to] > dist[edge.from] + edge.weight) {
                    dist[edge.to] = dist[edge.from] + edge.weight;
                    predecessor[edge.to] = edge.from;
                    updated = true;
                }
            }
        }

        if (!updated) break;
    }

    bool hasNegativeCycle = false;
    for (const Edge& edge : edges) {
        if (dist[edge.from] < INF) {
            if (dist[edge.to] > dist[edge.from] + edge.weight) {
                hasNegativeCycle = true;
                break;
            }
        }
    }

    if (hasNegativeCycle) {

```

```

        cout << "Граф содержит отрицательный цикл!" << endl;
        fill(dist.begin(), dist.end(), INF);
        fill(predecessor.begin(), predecessor.end(), -1);
    }

    return { dist, predecessor };
}

vector<int> getPath(const vector<int>& predecessor, int start, int target) const {
    vector<int> path;

    if (predecessor[target] == -1 && target != start) {
        return path;
    }

    for (int v = target; v != -1; v = predecessor[v]) {
        path.push_back(v);
        if (v == start) break;
    }

    reverse(path.begin(), path.end());

    if (!path.empty() && path[0] != start) {
        path.clear();
    }

    return path;
}

void printDetailedPaths(int start, const vector<double>& distances,
    const vector<int>& predecessor) const {
    cout << "\nДЕТАЛЬНЫЕ ПУТИ ОТ ВЕРШИНЫ " << start << ":\n";
    cout << "=====\n";

    for (int i = 0; i < vertices; i++) {
        cout << "До вершины " << i << " ";
        if (distances[i] == INF) {
            cout << "недостижима";
        }
        else if (i == start) {
            cout << distances[i] << ", путь: " << start;
        }
        else {
            cout << distances[i] << ", путь: ";
            vector<int> path = getPath(predecessor, start, i);
            if (!path.empty()) {
                for (size_t j = 0; j < path.size(); j++) {
                    cout << path[j];
                    if (j != path.size() - 1) cout << " -> ";
                }
            }
            else {
                cout << "не найден";
            }
        }
        cout << endl;
    }
}

void printInfo() const {
    cout << "Тип графа: ";
    if (isDirected) cout << "Ориентированный";
    else cout << "Неориентированный";

    cout << ", ";

    if (isWeighted) cout << "Взвешенный";

```



```

else cout << "Невзвешенный";

cout << "\nКоличество вершин: " << vertices << endl;
cout << "Количество ребер: " << edges.size() << endl;

if (isDirected) {
    cout << "Ребра:\n";
    for (const Edge& edge : edges) {
        cout << edge.from << " -> " << edge.to;
        if (isWeighted) {
            cout << " (вес: " << edge.weight << ")";
        }
        cout << endl;
    }
}
else {
    cout << "Ребра (неориентированные):\n";
    vector<vector<bool>>> printed(vertices, vector<bool>(vertices, false));
    for (const Edge& edge : edges) {
        int u = min(edge.from, edge.to);
        int v = max(edge.from, edge.to);
        if (!printed[u][v]) {
            cout << u << " -- " << v;
            if (isWeighted) {
                cout << " (вес: " << edge.weight << ")";
            }
            cout << endl;
            printed[u][v] = true;
        }
    }
}
cout << endl;
}

void saveToFile(const string& filename) const {
    ofstream file(filename);
    if (!file.is_open()) {
        cout << "Ошибка открытия файла для записи!" << endl;
        return;
    }

    file << vertices << endl;
    file << (isDirected ? 1 : 0) << endl;
    file << (isWeighted ? 1 : 0) << endl;
    file << edges.size() << endl;

    for (const Edge& edge : edges) {
        file << edge.from << " " << edge.to << " " << edge.weight << endl;
    }

    file.close();
    cout << "Граф сохранен в файл: " << filename << endl;
}

void loadFromFile(const string& filename) {
    ifstream file(filename);
    if (!file.is_open()) {
        cout << "Ошибка открытия файла!" << endl;
        return;
    }

    int v;
    file >> v;
    vertices = v;

    int directed, weighted;
    file >> directed >> weighted;

```

```

isDirected = (directed == 1);
isWeighted = (weighted == 1);

edges.clear();
int edgeCount;
file >> edgeCount;

for (int i = 0; i < edgeCount; i++) {
    int from, to;
    double weight;
    file >> from >> to >> weight;
    edges.push_back(Edge(from, to, weight));
}

file.close();
cout << "Граф загружен из файла: " << filename << endl;
}

void generateRandomGraph(int maxEdges = -1, double minWeight = -10.0, double maxWeight = 10.0) {
    edges.clear();

    if (maxEdges == -1) {
        maxEdges = vertices * (vertices - 1);
        if (!isDirected) maxEdges /= 2;
    }

    int edgeCount = rand() % maxEdges + 1;

    vector<vector<bool>> added(vertices, vector<bool>(vertices, false));

    for (int i = 0; i < edgeCount; i++) {
        int from, to;

        int attempts = 0;
        do {
            from = rand() % vertices;
            to = rand() % vertices;
            attempts++;
            if (attempts > vertices * vertices * 2) {
                edgeCount = i;
                break;
            }
        } while (from == to || added[from][to]);

        if (!isDirected) {
            added[from][to] = true;
            added[to][from] = true;
        }
        else {
            added[from][to] = true;
        }

        double weight = 1.0;
        if (isWeighted) {
            weight = minWeight + static_cast<double>(rand()) / (static_cast<double>(RAND_MAX) / (maxWeight - minWeight));
            weight = round(weight * 100.0) / 100.0;
        }

        addEdge(from, to, weight);
    }

    cout << "Сгенерирован случайный граф с " << edgeCount << " ребрами" << endl;
}

void generateCompleteRandomGraph() {
    clearScreen();

```

```

cout << "=== СЛУЧАЙНАЯ ГЕНЕРАЦИЯ ГРАФА ===\n\n";

int vertices;
bool directed, weighted;
char dirChoice, weightChoice;

cout << "Введите количество вершин (от 2 до 20): ";
cin >> vertices;
if (vertices < 2) vertices = 2;
if (vertices > 20) vertices = 20;

cout << "Ориентированный граф? (y/n): ";
cin >> dirChoice;
directed = (dirChoice == 'y' || dirChoice == 'Y');

cout << "Взвешенный граф? (y/n): ";
cin >> weightChoice;
weighted = (weightChoice == 'y' || weightChoice == 'Y');

setVertices(vertices);
setDirected(directed);
setWeighted(weighted);

double minWeight = -10.0;
double maxWeight = 10.0;
if (weighted) {
    cout << "Минимальный вес ребра (по умолчанию -10.0): ";
    cin >> minWeight;
    cout << "Максимальный вес ребра (по умолчанию 10.0): ";
    cin >> maxWeight;

    if (minWeight > maxWeight) {
        swap(minWeight, maxWeight);
        cout << "Диапазоны весов поменяны местами." << endl;
    }
}

int densityChoice;
cout << "\nВыберите плотность графа:\n";
cout << "1. Разреженный (20% от максимального количества ребер)\n";
cout << "2. Средней плотности (50% от максимального количества ребер)\n";
cout << "3. Плотный (80% от максимального количества ребер)\n";
cout << "4. Случайная плотность\n";
cout << "Ваш выбор: ";
cin >> densityChoice;

int maxPossibleEdges;
if (directed) {
    maxPossibleEdges = vertices * (vertices - 1);
}
else {
    maxPossibleEdges = vertices * (vertices - 1) / 2;
}

int maxEdges;
switch (densityChoice) {
    case 1: maxEdges = maxPossibleEdges * 0.2; break;
    case 2: maxEdges = maxPossibleEdges * 0.5; break;
    case 3: maxEdges = maxPossibleEdges * 0.8; break;
    case 4: maxEdges = maxPossibleEdges; break;
    default: maxEdges = maxPossibleEdges * 0.5; break;
}

if (maxEdges < 1) maxEdges = 1;

generateRandomGraph(maxEdges, minWeight, maxWeight);

```

```

cout << "\nГраф успешно сгенерирован со следующими параметрами:\n";
cout << "- Вершин: " << vertices << endl;
cout << "- Тип: " << (directed ? "Ориентированный" : "Неориентированный") << endl;
cout << "- Веса: " << (weighted ? "Взвешенный" : "Невзвешенный") << endl;
if (weighted) {
    cout << "- Диапазон весов: [" << minWeight << ", " << maxWeight << "]" << endl;
}
cout << "- Плотность: ";
switch (densityChoice) {
case 1: cout << "Разреженный"; break;
case 2: cout << "Средней плотности"; break;
case 3: cout << "Плотный"; break;
case 4: cout << "Случайная"; break;
}
cout << endl;
}

void inputFromKeyboard() {
    edges.clear();
    int edgeCount;

    cout << "Введите количество ребер: ";
    cin >> edgeCount;

    cout << "Введите ребра в формате: из_вершины в_вершину [вес]" << endl;
    cout << "Вершины нумеруются от 0 до " << vertices - 1 << endl;

    for (int i = 0; i < edgeCount; i++) {
        int from, to;
        double weight = 1.0;

        cout << "Ребро " << i + 1 << ": ";
        cin >> from >> to;

        if (isWeighted) {
            cin >> weight;
        }

        if (from < 0 || from >= vertices || to < 0 || to >= vertices) {
            cout << "Ошибка: вершины должны быть в диапазоне 0-" << vertices - 1 << endl;
            i--;
            continue;
        }

        addEdge(from, to, weight);
    }
}
}

```